

Glossário Técnico da Obra

U · N · I

I FUNDAMENTOS DE SISTEMAS EMBARCADOS

Sistemas Embarcados

Sistemas computacionais projetados para executar funções dedicadas dentro de um produto ou dispositivo maior, com restrições de consumo, custo e tamanho. Diferem dos computadores de propósito geral por terem função fixa, otimização específica e operação sem intervenção humana direta. No contexto desta obra, abrangem desde microcontroladores como o ESP32-S3 (K10) até computadores Linux como o RK3308 (M10). [Cap. 1–4](#).

Veja também: [Linux Embarcado](#) · [Microcontrolador](#) · [SoC](#)

Computação Física

Disciplina que combina hardware, software e sensores/atuidores para criar sistemas que interagem com o mundo físico. É o campo onde a engenharia embarcada encontra o mundo real: luzes, motores, displays, sons e gestos. O UNIHIKER é, por design, uma plataforma de computação física — integra display, sensores e GPIO em uma única placa. [Cap. 2, Seção 2.4](#).

Veja também: [Edge AI](#) · [Sistemas Embarcados](#) · [STEAM](#)

Linux Embarcado

Versão do sistema operacional Linux adaptada para operar em dispositivos com recursos limitados, tipicamente SoCs ARM sem BIOS/UEFI. Inclui kernel customizado via Device Tree, bootloader U-Boot, sistema de init systemd e distribuição minimalista (Debian, Yocto ou Buildroot). O UNIHIKER M10 roda Debian 12 Bookworm sobre kernel 5.10 LTS. Diferente do Linux de desktop por ter boot via U-Boot (não GRUB), device tree (não ACPI) e init via systemd com serviços paralelos. [Cap. 4, Seção 4.5](#).

Veja também: [Device Tree](#) · [Bootloader](#) · [Kernel](#)

Firmware

Programa gravado permanentemente em memória não-volátil (flash, eMMC, ROM) que controla o hardware de baixo nível. Inclui bootloader, kernel, device tree e sistema de arquivos. No K10, o firmware inclui MicroPython + bootloader de 2 estágios; no M10, inclui U-Boot + kernel Linux + DTB + rootfs ext4. [Cap. 3–4](#).

Veja também: [Bootloader](#) · [OTA](#) · [Device Tree](#)

Bootloader

Programa responsável por inicializar o hardware e carregar o sistema operacional. Em sistemas embarcados Linux, o padrão é o U-Boot, que executa em estágios: SPL (inicializa DDR3/PMU), mainloader (carrega kernel + DTB para RAM). Diferente do GRUB (que depende de BIOS/UEFI), o U-Boot inicializa o hardware diretamente. [Cap. 4, Seção 4.4](#).

Veja também: [Firmware](#) · [Device Tree](#) · [Linux Embarcado](#)

Device Tree

Estrutura de dados binária (DTB) que descreve o hardware específico de uma placa ao kernel Linux genérico. Substitui a antiga prática de compilar configurações de placa dentro do kernel. O arquivo-fonte DTS do M10 (rk3308-m10.dts) deriva do rk3308-evb.dts da Rockchip com customizações da DFRobot. [Cap. 4, Seção 4.4.2](#).

Veja também: [Linux Embarcado](#) · [Kernel](#) · [Bootloader](#)

Microcontrolador

Circuito integrado que combina CPU, memória (SRAM, Flash) e periféricos (GPIO, ADC, UART, I²C, SPI) em um único chip. Projetado para aplicações embarcadas de baixo consumo e custo reduzido. O ESP32-S3 do K10 é um microcontrolador; o RK3308 do M10 é um SoC (mais potente, com MMU e Linux). [Cap. 3, Seção 3.2](#).

Veja também: [SoC](#) · [Sistemas Embarcados](#)

SoC (System-on-Chip)

Circuito integrado que integra CPU, GPU, controladores de memória, periféricos e aceleradores em um único die. Diferente do microcontrolador, o SoC tipicamente suporta sistema operacional completo (Linux) e tem MMU. O RK3308 (M10) é um SoC ARM Cortex-A35 quad-core; o ESP32-S3 (K10) é um microcontrolador Xtensa. [Cap. 3–4](#).

Veja também: [Microcontrolador](#) · [Linux Embarcado](#)

II INTELIGÊNCIA ARTIFICIAL NA BORDA

Edge AI

Execução de inferência de modelos de inteligência artificial diretamente no dispositivo (edge), sem depender de nuvem. Reduz latência, protege privacidade e funciona offline. No K10, Edge AI é feita via TinyML (modelos < 100 KB no ESP32-S3); no M10, via TensorFlow Lite (modelos até ~500 MB com Linux). [Cap. 2, 3, 4, 6](#).

Veja também: [TinyML](#) · [Inferência](#) · [Edge Computing](#)

TinyML

Subcampo do machine learning que treina e executa modelos em microcontroladores com recursos severamente limitados (KB de RAM, MHz de clock, mW de potência). Usa quantização int8, pruning e arquiteturas leves (MobileNet, autoencoders). No K10, TinyML permite detecção de anomalias em ~10ms. [Cap. 3, Seção 3.9](#); [Cap. 6, Seção 6.7](#).

Veja também: [Edge AI](#) · [Inferência](#) · [Quantização](#)

Inferência

Processo de aplicar um modelo de IA treinado a novos dados para produzir uma previsão, classificação ou detecção. Diferente do treinamento (que exige GPU e grandes datasets), a inferência pode rodar em dispositivos edge com recursos limitados. No K10, inferência via TFLite Micro; no M10, via TFLite sobre Linux. [Cap. 3–4, 6, 11](#).

Veja também: [Edge AI](#) · [TinyML](#) · [Pipeline](#)

Edge Computing

Paradigma de computação que processa dados próximo à fonte (sensor, dispositivo), reduzindo latência, largura de banda e dependência de nuvem. A pirâmide da computação (Cap. 6) organiza-se em camadas: sensor → edge (K10) → gateway/fog (M10) → cloud. Edge Computing é a base que viabiliza Edge AI. [Cap. 2, 6](#).

Veja também: [Edge AI](#) · [Fog Computing](#) · [Pirâmide da Computação](#)

Fog Computing

Camada intermediária entre Edge e Cloud, tipicamente executada em gateways Linux (como o M10). Agrega dados de múltiplos dispositivos edge, executa processamento local, armazena temporariamente e encaminha à nuvem apenas informações relevantes. Reduz carga na nuvem e mantém operação durante falhas de conectividade. [Cap. 6, Seção 6.4](#).

Veja também: [Edge Computing](#) · [Gateway](#) · [Pirâmide da Computação](#)

Quantização

Técnica de otimização que reduz a precisão dos pesos de um modelo de IA de float32 para int8 ou menor, diminuindo tamanho e acelerando inferência com perda mínima de acurácia. Essencial para TinyML em microcontroladores. O Edge Impulse automatiza quantização durante o treinamento. [Cap. 3, Seção 3.9](#); [Cap. 6, Seção 6.7](#).

Veja também: [TinyML](#) · [Inferência](#)

III ARQUITETURA DE SISTEMAS

Arquitetura de Sistemas Inteligentes

Disciplina que projeta a estrutura completa de soluções de IoT e Edge AI, definindo camadas (sensor → edge → gateway → cloud), protocolos, escalabilidade, segurança e fluxos de dados. No Cap. 6, são apresentadas nove arquiteturas de referência por domínio (IoT industrial, agricultura, smart city, robótica, saúde, Indústria 4.0, ambiente, etc.). [Cap. 6](#).

Veja também: [Pirâmide da Computação](#) · [Trade-off](#) · [Escalabilidade](#)

Pirâmide da Computação

Modelo hierárquico que organiza a computação em nove camadas, do sensor físico (base) à tomada de decisão estratégica (topo). Volume de dados diminui com a altura; latência e exigência computacional aumentam. K10 opera na camada Edge; M10 na camada Gateway/Fog; cloud no topo. [Cap. 6, Seção 6.2](#).

Veja também: [Edge Computing](#) · [Fog Computing](#)

Trade-off

Decisão de engenharia em que a escolha de uma vantagem implica sacrificar outra. Não existe almoço gratuito: cada decisão técnica (K10 vs M10, Linux vs RTOS, cloud vs edge) envolve trade-offs de desempenho, consumo, custo, complexidade e escalabilidade. O Cap. 5 é dedicado ao framework de trade-offs entre K10 e M10. [Cap. 5, 8](#).

Veja também: [Arquitetura de Sistemas Inteligentes](#) · [Matriz de Decisão](#)

Gateway

Dispositivo intermediário que agrega dados de múltiplos dispositivos edge, executa processamento local e encaminha à nuvem. Tipicamente roda Linux (como o M10) com Node-RED, InfluxDB e broker MQTT. O gateway é a camada Fog da pirâmide de computação. [Cap. 6, 11](#).

Veja também: [Fog Computing](#) · [Pirâmide da Computação](#)

Escalabilidade

Capacidade de um sistema crescer de poucos dispositivos (protótipo) a dezenas de milhares (produção nacional) sem reescrever arquitetura. No Cap. 8, o UEF define seis níveis de escalabilidade (1 → 10 → 100 → 1.000 → 10.000 → 100.000+), cada um com desafios específicos de broker, armazenamento e operação. [Cap. 6, 8](#).

Veja também: [Arquitetura de Sistemas Inteligentes](#) · [UEF](#)

Pipeline

Sequência de etapas de processamento conectadas em série, onde a saída de uma etapa é a entrada da próxima. Pode ser pipeline de boot (BootROM → SPL → U-Boot → Kernel → init), pipeline de dados (sensor → pré-processamento → inferência → pós-processamento → cloud) ou pipeline de CI/CD (build → test → package → deploy). [Cap. 4, 6, 9](#).

Veja também: [Bootloader](#) · [Inferência](#) · [CI/CD](#)

Middleware

Camada de software que abstrai a comunicação entre aplicações e hardware/sistema operacional. Node-RED é middleware visual de IoT; Mosquitto é middleware de mensagens MQTT; InfluxDB é middleware de séries temporais. O middleware reduz complexidade e acelera desenvolvimento. [Cap. 6, 9](#).

Veja também: [Framework](#) · [Pipeline](#)

Digital Twin

Réplica digital de um sistema físico que sincroniza em tempo real via sensores IoT. Permite simulação, predição e otimização sem interferir no sistema real. No Cap. 11, o Caso 10 (Smart City) menciona digital twins de infraestrutura urbana como evolução natural de sistemas IoT maduros. [Cap. 7, 11, 12](#).

Veja também: [Edge Computing](#) · [Observabilidade](#)

IV ENGENHARIA DE SISTEMAS E METODOLOGIA

Engenharia de Sistemas

Disciplina que estrutura o ciclo de vida completo de sistemas complexos, da concepção à aposentadoria. Definida pela INCOSE e pelas normas ISO 15288 (processos de ciclo de vida) e ISO 25010 (qualidade de software). Esta obra aplica engenharia de sistemas ao domínio de plataformas embarcadas inteligentes. [Cap. 8, 12](#).

Veja também: [UEF](#) · [Framework](#)

UEF (UNIHAKER Engineering Framework)

Metodologia proprietária apresentada no Cap. 8, com 15 etapas sequenciais que estruturam todo o ciclo de vida de projetos UNIHAKER: Ideia → Requisitos → Arquitetura → Plataforma → Sensores → Protocolos → IA → Protótipo → Validação → Piloto → Produção → Escala → Manutenção → Operação → Evolução. Cada etapa tem entregável e checkpoint de qualidade. [Cap. 8](#).

Veja também: [Engenharia de Sistemas](#) · [Framework](#) · [Trade-off](#)

Framework

Estrutura conceitual e metodológica que organiza e padroniza um processo. Diferente de uma biblioteca (que fornece funções), um framework define o fluxo, os entregáveis e os checkpoints. O UEF é um framework de engenharia; o TFLite é um framework de inferência; o Node-RED é um framework de fluxos visuais. [Cap. 8, 9](#).

Veja também: [UEF](#) · [Middleware](#)

Matriz de Decisão

Ferramenta de engenharia que avalia alternativas técnicas em múltiplos critérios ponderados, produzindo uma pontuação comparativa. No Cap. 5, a matriz K10 vs M10 avalia 16 critérios (CPU, RAM, consumo, custo, IA, etc.) com pontuação 0–10, permitindo decisão fundamentada e documentada. [Cap. 5, Seção 5.5](#).

Veja também: [Trade-off](#) · [Árvore de Decisão](#)

Real-Time (Tempo Real)

Propriedade de sistemas que garantem resposta dentro de prazo determinístico, independentemente da carga. Sistemas hard real-time (controle de motor) nunca podem perder deadlines; soft real-time (streaming de áudio) toleram variações. O K10 com FreeRTOS oferece tempo real; o M10 com Linux não é determinístico. [Cap. 3, 5](#).

Veja também: [Sistemas Embarcados](#) · [RTOS](#)

V DEVOPS E OPERAÇÃO

DevOps

Cultura e conjunto de práticas que integram desenvolvimento (Dev) e operação (Ops) através de automação, CI/CD, monitoramento e feedback contínuo. No contexto embarcado, DevOps inclui Git Flow para firmware, CI para build automatizado, OTA para deploy, observabilidade para operação e rollback para recuperação. [Cap. 9](#).

Veja também: [CI/CD](#) · [OTA](#) · [Observabilidade](#)

CI/CD

Pipeline automatizado que integra código continuamente (CI — build + test a cada commit) e entrega em produção (CD — deploy automatizado com aprovação). No Cap. 9, o CI/CD embarcado inclui lint, build, testes unitários, HIL, package e deploy staging. Ferramentas: GitHub Actions, GitLab CI. [Cap. 9, Seção 9.5](#).

Veja também: [DevOps](#) · [Pipeline](#) · [OTA](#)

OTA (Over-The-Air Update)

Atualização remota de firmware via rede sem fio, sem acesso físico ao dispositivo. OTA profissional inclui assinatura criptográfica, rollback automático (A/B partitions), verificação de integridade e deploy gradual (canário). Essencial para sistemas em escala que não podem ser atualizados manualmente. [Cap. 9, Seção 9.6](#).

Veja também: [DevOps](#) · [Firmware](#) · [Rollback](#)

Observabilidade

Capacidade de entender o estado interno de um sistema a partir de suas saídas externas (logs, métricas, traces). Diferente de monitoramento (que verifica se algo está funcionando), observabilidade permite diagnosticar por que algo falhou. Stack típica: Prometheus (métricas), Loki (logs), Grafana (dashboards), Jaeger (traces). [Cap. 9, Seção 9.7](#).

Veja também: [DevOps](#) · [Digital Twin](#)

Rollback

Mecanismo que reverte automaticamente para a versão anterior do firmware quando a nova versão falha (kernel panic, crash de aplicação, perda de conectividade). Implementado via partições A/B: o OTA grava na partição inativa; se o health-check falhar após reboot, o bootloader volta para a partição ativa anterior. [Cap. 9, Seção 9.6](#).

Veja também: [OTA](#) · [Bootloader](#)

Secure Boot

Cadeia de verificação criptográfica que garante que apenas firmware assinado execute no dispositivo, do BootROM ao kernel. Cada estágio verifica a assinatura do próximo antes de transferir controle. Previne firmware malicioso. Deve ser projetado desde a primeira versão — não pode ser adicionado depois. [Cap. 9, Seção 9.10](#).

Veja também: [Bootloader](#) · [Firmware](#)

SBOM (Software Bill of Materials)

Inventário completo de todos os componentes de software (bibliotecas, dependências, versões, licenças) de um sistema. Essencial para auditoria de segurança e conformidade. Gerado automaticamente no pipeline CI/CD. [Cap. 9, Seção 9.10](#).

Veja também: [CI/CD](#) · [DevOps](#)

VI INSTRUMENTAÇÃO E COMPONENTES

Sensor

Dispositivo que converte um fenômeno físico (temperatura, pressão, luz, aceleração) em sinal elétrico mensurável. É o primeiro elo da cadeia de instrumentação. Nem todo sensor é um transdutor completo; nem todo transdutor é um instrumento. O módulo Gravity BME280 é um instrumento (sensor + transdutor + condicionamento + conector). [Cap. 10, Seção 10.1](#).

Veja também: [Transdutor](#) · [Calibração](#)

Transdutor

Elemento que converte uma forma de energia em outra, tipicamente fenômeno físico em sinal elétrico. O sensor detecta; o transdutor converte; o instrumento integra sensor + transdutor + condicionamento + interface. Exemplo: termistor é sensor+transdutor; módulo Gravity com termistor é instrumento. [Cap. 10, Seção 10.1](#).

Veja também: [Sensor](#) · [Atuador](#)

Atuador

Dispositivo que converte sinais elétricos em ação física (motor, relé, servomecanismo, solenoide, display, buzzer). É o inverso do sensor: enquanto o sensor lê o mundo, o atuador atua sobre ele. Seleção de atuador considera torque, velocidade, consumo, precisão e compatibilidade com K10/M10. [Cap. 10, Seção 10.7](#).

Veja também: [Sensor](#) · [Computação Física](#)

Calibração

Processo de ajustar a resposta de um sensor a um padrão de referência conhecido, eliminando erros sistemáticos. Sensores baratos driftam em semanas; sensores industriais mantêm calibração por anos. Em ETA/ETE (Cap. 11), sensores de pH baratos falham em 3 meses; Atlas Scientific mantém calibração por 12+ meses. [Cap. 10, 11](#).

Veja também: [Sensor](#) · [Precisão](#) · [TCO](#)

TCO (Total Cost of Ownership)

Soma de CAPEX (investimento inicial) e OPEX (custo operacional recorrente) ao longo do ciclo de vida do sistema. Inclui hardware, sensores, instalação, cloud, equipe, manutenção e substituição. No Cap. 5, TCO de 3 anos é analisado para K10 vs M10; no Cap. 11, TCO de 5 anos é calculado para cada estudo de caso. [Cap. 5, 10, 11](#).

Veja também: [CAPEX](#) · [OPEX](#) · [Calibração](#)